

Why Documentation?

Types, audiences, standards, and the engineering culture that turns documentation from liability into a force multiplier.

REQUIRED READING POINTER FOR THIS SESSION

Bhatti et al. — Ch. 1 „Understanding your audience“ (S. 1–21) · **Chinchilla** — Ch. 1 „The Why, Who, How of Tech Writing“ (S. 1–8)

Key concepts: „curse of knowledge“ (Bhatti S. 3) · „writer-in-the-middle“ (Chinchilla S. 5) · end-end-users (Chinchilla S. 7)

Ph.D. Batnyam Battulga, Dipl.-Ing., Dipl.-Inf.

National University of Mongolia · Dept. of Software Engineering

LECTURE · 90 MIN

Foundations & Culture

Bhatti Ch. 1 S. 1–21 + Chinchilla Ch. 1 S. 1–8

Followed by a 90-min Seminar on the myth
„Good code documents itself“.

Today's Agenda

90-minute lecture, six blocks, one exit question — structured by Bhatti's „curse-of-knowledge“ concept (Ch. 1 S. 3).

00–10

Opening: “Which doc saved you? Which failed you?”

Anker: Bhatti Ch. 1 S. 4–14 · Chinchilla Ch. 1 S. 5–7

Card-collection, lived experiences, surfacing biases.

Activity: silent brainstorm on Post-its.

10–30

Four Documentation Types

Anker: Bhatti Ch. 2 S. 23–44 (Planning) · Chinchilla Ch. 2 S. 9–24 (Understanding Different Types)

User documentation · Developer documentation · Operations documentation · Process documentation · Living documentation.

30–55

Purpose & Audiences

Anker: Bhatti Ch. 1 S. 15–17 (Personas + Journey Maps) · Chinchilla Ch. 1 S. 2–7 (Why/Who/How)

Onboarding · Maintenance · Knowledge transfer · Compliance · Audit · Handover — cost/benefit calculation.

55–80

Standards & Culture

Anker: Bhatti Ch. 8 S. 133–146 (Anti-Patterns / Culture) · Chinchilla Ch. 6 S. 69–85 (Docs-as-Code)

Overview: IEEE 830, ISO/IEC/IEEE 29148, Diátaxis. Anti-patterns: tribal knowledge, secrecy, hero culture.

80–90

Outlook: The Course Project (Leitprojekt)

Anker: Bhatti Ch. 2 S. 41 (Planning wraps) + Chinchilla Ch. 5 S. 53–55 (Scoping)

System selection criteria + self-study task (workspace setup). Homework: criteria list & workspace URL by Friday.

Week guide: Bhatti et al. Ch. 1 S. 1–21 + Ch. 2 S. 23–44 · Chinchilla Ch. 1 S. 1–8 + Ch. 2 S. 9–24 · Begleitend: Chinchilla Ch. 5 S. 53–63 (Process)



Which documentation has recently **saved** you — and which has **failed** you?

Cf. Bhatti Ch. 1 S. 3 — „The curse of knowledge“: 51% tappers vs. 2.5% listeners (Newton 1990).

Two cards. C1: a moment a doc rescued your afternoon. C2: a moment a missing or wrong doc cost you half a day. Write two short sentences, one per card. We will cluster them on the wall.

WHY WE ASK · *Chinchilla Ch. 1 S. 5 „writer-in-the-middle“*

Lived experience is the strongest entry into a topic. Without it, “documentation” stays an abstract school requirement. With two minutes of concrete memory, it becomes an engineering problem you have already seen.

Key concepts on this slide: Bhatti Ch. 1 S. 3 (Curse-of-Knowledge) · Chinchilla Ch. 1 S. 5 (Writer-in-the-middle) + S. 7 (End-end-users)

The Four Documentation Types

Every artefact you will produce in this course falls into one of these four buckets — and mixes them only at your peril. Bhatti's four-fold "Code comments / READMEs / Getting started / Conceptual / Procedural / Reference" (Ch. 2, S. 23–44) answers the same question conceptually; Chinchilla's four-fold "Getting Started / Tutorial / Reference / API" (Ch. 2, S. 9–24) answers it operationally.

01

User Documentation

Bhatti Ch. 2 S. 27–29 · Chinchilla Ch. 2 S. 10–14

Audience: end users, customers, integrators. Examples: User guide, tutorials, FAQ, release notes, error-message texts.

02

Developer Documentation

Bhatti Ch. 2 S. 25 · Ch. 5 S. 83–99 · Chinchilla Ch. 2 S. 15–23

Audience: engineers who extend or maintain the code. Examples: API reference, architecture (arc42, C4), ADRs, README, code comments.

03

Operations Documentation

Bhatti Ch. 1 S. 9 · Ch. 8 S. 133–146 · Chinchilla Ch. 1 S. 3

Audience: SRE, ops, on-call engineers. Examples: Runbooks, deployment guides, incident postmortems, SLO definitions.

04

Process Documentation

Bhatti Ch. 8 S. 138 · Ch. 9 S. 162 · Chinchilla Ch. 1 S. 2

Audience: PMs, auditors, new joiners. Examples: workflow, decision logs, risk register, change log, retrospectives.

LIVING DOCUMENTATION · *Bhatti Ch. 2 S. 41 „Planning your documentation“ + Chinchilla Ch. 2 § „Summary“ (S. 24)*

Modern practice blends these — e.g. an OpenAPI spec is at once **developer** docs and a piece of **operations** truth. But before you mix types, finish one clean example of each. W5–W7 of this course.

Required reading pointer: Bhatti et al. Ch. 2 „Planning your documentation“ S. 23–44 · Ch. 1 S. 1–21 (Curse-of-Knowledge) · Chinchilla Ch. 2 S. 9–24 · Ch. 1 S. 2–7 (Writer-in-the-middle)

Purpose & Audiences

Documentation is an answer to a cost question. Answer it first, then write — anchored in Bhatti Ch. 9 S. 149.

AUDIENCE × PURPOSE MATRIX · after Bhatti Ch. 1 S. 15-21 (Persona + Journey-Map)

Audience	Primary purpose	Where it fails (Bhatti Anti-Pattern)
New joiner	Onboarding within one week	Outdated setup steps <i>Ch. 1 S. 9 (Support-Tickets)</i>
Maintainer	Safe change in 1 day	Missing ADRs <i>Ch. 4 ADRs / W4</i>
Operator (SRE)	Restore service during incident	No runbook <i>Ch. 8 S. 133-146 (Feedback)</i>
Auditor / regulator	Trace decision and process	No change log <i>Ch. 9 S. 162 (Metrics)</i>
External partner	Self-serve integration	Inconsistent API examples <i>Chinchilla Ch. 2 S. 18-22</i>

COST / BENEFIT REASONING

Bhatti Ch. 9 S. 149 — „A document is good when it fulfills its purpose“

Documentation is paid in two currencies.

Author time — every page costs an engineer-day to write and review.

Reader time — every unclear page costs a future reader an hour.

Author cost is paid once. Reader cost compounds. The bias toward omission comes from forgetting that the next reader may be yourself in six months.

Required reading pointer: Bhatti et al. Ch. 1 S. 15-21 · Ch. 8 S. 133-146 · Ch. 9 S. 149 · Chinchilla Ch. 1 S. 2-7 · Ch. 2 S. 18-22

Standards & Culture

Three frameworks you will use this semester — and the cultural traps every framework can fall into.

REQUIREMENTS STANDARD

IEEE 830 / ISO 29148

Anchor: Bhatti Ch. 2 S. 23–44 · Chinchilla Ch. 2 S. 24

Structure for a Software Requirements Specification.
Required for safety-critical and regulated systems. We use it in Weeks 2 & 3.

ARCHITECTURE STANDARD

arc42 & C4

Anchor: Bhatti Ch. 2 S. 41 · Ch. 6 S. 108–117 · Chinchilla Ch. 2 S. 22–23

arc42 gives the prose template, C4 the diagram levels. We combine both in Week 4. ADRs sit inside arc42 as captured decisions.

USER-DOCUMENTATION FRAMEWORK

Diátaxis

Anchor: Bhatti Ch. 2 S. 31–32 (10-step tutorial) · Chinchilla Ch. 2 S. 12–14 (Tutorial)

Four documentation modes — Tutorial, How-to, Reference, Explanation — share one site, no mode-blending. The structure of Week 7.

ANTI-PATTERNS WE WILL REFUSE THIS SEMESTER · Bhatti Ch. 8 S. 133–146 + Chinchilla Ch. 3

1. Tribal knowledge — “Ask Ali — he knows this module.” [Bhatti Ch. 1 S. 9]
2. Hero culture — one person becomes *the* single point of failure. [Bhatti Ch. 4 S. 79]
3. Cargo-cult standards — pasting the IEEE 830 template without the discipline behind it. [Bhatti Ch. 3 S. 58 + Chinchilla Ch. 5 S. 53–55]

CULTURE WE WILL BUILD · Chinchilla Ch. 6 S. 69–85

1. Documentation as code — reviewable, versioned, tested.
2. Reader-first writing — structure for the question, not the answer.
3. Defensible omissions — if you choose to omit, write down why.

Required reading pointer: Bhatti et al. Ch. 2 S. 23–44 + Ch. 8 S. 133–146 · Chinchilla Ch. 2 S. 9–24 + Ch. 3 + Ch. 6 S. 69–85

CONCEPT TEST

Vote individually · 30-second peer defence · re-vote · reveal

QUESTION 1 / 1 · Bhatti Ch. 1 S. 4 „Defining your users' goals“

A junior backend engineer joins your team next Monday. Which single artefact will most reduce their time to first merged pull request?

A

Anchor: Bhatti Ch. 2 S. 23–44 (SRS template)

A 60-page Software Requirements Specification (IEEE 830).

B

Anchor: Bhatti Ch. 2 S. 27–29 (READMEs) + Chinchilla Ch. 2 S. 10–12

A 2-page README with build steps, test command, and one worked example.

C

Anchor: Bhatti Ch. 4 S. 41 (Planning) + Chinchilla Ch. 2 S. 22–23

A 30-slide architecture-deck with C4 diagrams and three ADRs.

D

Anchor: Bhatti Ch. 1 S. 9 (Support Tickets) + Anti-Pattern 01

A list of “the five things you always forget to set up” (Post-it notes from the last on-call rotation).

REVEAL ANSWER — AFTER VOTES · Bhatti Ch. 9 S. 149 as evaluation framework

B (the README) is correct.

Reduce time-to-first-PR to a half-day. SRS = compliance · Architecture = senior handover · Post-its = Tribal Knowledge (Ch. 1 S. 9).

Week anchor: Bhatti Ch. 1 S. 4 + S. 9 + Ch. 9 S. 149 · Chinchilla Ch. 1 S. 5 + Ch. 5 S. 60–63

Three Anti-Patterns, in Concrete Form

Each pattern is recognisable in code review within ten minutes — if you know what to look for.

PATTERN 01 · TRIBAL KNOWLEDGE

Anchor: Bhatti Ch. 1 S. 9 (Support Tickets) + Ch. 8 S. 133–146

“Only Ali can deploy on Fridays.”

Symptom: The codebase has no runbook; production knowledge lives in one head.

DIAGNOSTIC

Search the repo for “see Ali”. Count Slack threads that resolve with “I’ll just do it”.

COUNTER-MEASURE

Pair-program the next on-call rotation (Ch. 8 S. 141 Triaging feedback). Then write down what you learned as a runbook.

PATTERN 02 · HERO CULTURE

Anchor: Bhatti Ch. 4 S. 79 (Giving good feedback) + Ch. 8 S. 138 (User Council)

One person becomes a single point of failure.

Symptom: One engineer migration-run-mastered the system; the team quietly de-prioritises documentation.

DIAGNOSTIC

Ask: “Who would notice if this person left next Friday?” (Chinchilla Ch. 1 S. 7 „end-end-users“)

COUNTER-MEASURE

Mandatory docs as part of “definition of done” (Week 9, Bhatti Ch. 9 S. 162).

PATTERN 03 · CARGO-CULT STANDARDS

Anchor: Bhatti Ch. 3 S. 58 (Most-important-info-first) + Chinchilla Ch. 5 S. 53–55 (Scoping)

Standards as labels, not tools.

Symptom: 60-page SRS with empty requirement text, written because “the customer asked”. Audit-trail in name only.

DIAGNOSTIC

Pick one requirement. Try to write a black-box test from it. If you cannot, it is decoration.

COUNTER-MEASURE

Write the test first in Week 3 (M3) — if there is no test, the requirement is not yet specified.

Required reading pointer: Bhatti et al. Ch. 1 S. 9 · Ch. 3 S. 58 · Ch. 4 S. 79 · Ch. 8 S. 138/141 · Ch. 9 S. 162 · Chinchilla Ch. 1 S. 7 + Ch. 5 S. 53–55

Good vs. Bad Documentation Culture

Seven engineering pairs that separate teams that scale from teams that stall.

DIMENSION	BAD CULTURE	GOOD CULTURE + Bhatti/Chinchilla anchor
Target reader	Every developer, “including the new one”, “including the auditor”.	One named reader per document; a different document for each. <i>Bhatti Ch. 1 S. 15-21 (Personas)</i>
Update discipline	Updated “when someone has time”.	PR-Review inkludiert Doc-Diff; CI fails on broken links. <i>Bhatti Ch. 8 S. 133-146 + Chinchilla Ch. 6 S. 69-71 (Docs-as-code)</i>
Ownership	Implicit — “the team owns the README”.	Named owner per section — rotation visible on the workspace. <i>Bhatti Ch. 8 S. 138 (User Council)</i>
Style	Each contributor’s voice.	Style guide = Google Developer Style + your project termbase. <i>Bhatti Ch. 4 S. 67-81 + Chinchilla Ch. 3 S. 28</i>
Failure handling	Hot-fix in code; doc left out of date.	Incident postmortem has a doc-update item as required deliverable. <i>Bhatti Ch. 9 S. 160 (User goals & metrics)</i>
Newcomer time	“Two to four weeks”.	“Two to four days” — measured, not assumed. <i>Bhatti Ch. 1 S. 4-7 + Chinchilla Ch. 5 S. 60-63</i>
Traceability	Decisions in chat, lost in two weeks.	ADRs (Week 4) + dated changelog embedded in every artefact. <i>Bhatti Ch. 8 S. 141 + Chinchilla Ch. 6 S. 76 (YAML)</i>

Required reading pointer per row: Bhatti Ch. 1+4+8+9 + Chinchilla Ch. 3+5+6 — each good-culture cell carries a book reference.

Case Study — Stripe API Documentation

What an industry-grade reference site looks like, and the six design choices that make it work.

stripe.com/docs/api/charges — GET

Create a charge

POST /v1/charges

```
// Example request
curl https://api.stripe.com/v1/charges \
-u sk_test_4eC39HqLyjWDarjtT1zdp7dc: \
-d amount=2000 -d currency="usd"
```

Real cURL examples · Language tabs (cURL, Ruby, Python, Node, Go, Java, PHP, .NET) · Inline response JSON · Error taxonomy · Version switcher.

SIX DESIGN CHOICES · WITH ANCHORS

- 1. Reader-first ordering — request, params, return, errors.** [\[Bhatti Ch. 3 S. 58\]](#)
- 2. Code before prose — the cURL block is the first object.** [\[Bhatti Ch. 5 S. 92–94, Usable / Trustworthy\]](#)
- 3. Eight language tabs — one click, full example in your stack.** [\[Chinchilla Ch. 7 S. 88–92, Code examples\]](#)
- 4. Explicit error taxonomy — ahead of success borders.** [\[Bhatti Ch. 5 S. 96–98 + Chinchilla Ch. 4 S. 50\]](#)
- 5. Versioning visible — reader knows which API version is current.** [\[Chinchilla Ch. 6 S. 79, Rendering\]](#)
- 6. Changelog at every endpoint — what changed, when, why.** [\[Bhatti Ch. 8 S. 133–146 + Chinchilla Ch. 6 S. 76, YAML\]](#)

Required reading pointer: [Bhatti et al. Ch. 3 S. 58 + Ch. 5 S. 92–98 + Ch. 8 S. 133–146](#) · [Chinchilla Ch. 2 S. 18–22 + Ch. 4 S. 50 + Ch. 6 S. 76/79 + Ch. 7 S. 88–92](#)

Three Takeaways

What you should leave the room with and what you should bring to your first workspace session.

01

Bhatti Ch. 1 S. 15–21

A doc always has one named reader

Different readers = different documents. SRS for compliance, README for newcomers, runbook for ops. Blending them is the most common cause of unusable docs.

02

Chinchilla Ch. 1 S. 7 + Bhatti Ch. 9 S. 149

Reader cost compounds; author cost is paid once

The bias toward omission comes from ignoring the next reader. Measure new-joiner time as your documentation KPI.

03

Bhatti Ch. 2 S. 23–44 + Chinchilla Ch. 6 S. 69–85

Standards are tools, not labels

IEEE 830, arc42, Diátaxis — use them as scaffolds, not as decorations. The discipline behind the structure is what matters.

TODAY'S INDIVIDUAL COMMITMENT · Chinchilla Ch. 1 S. 5 + Bhatti Ch. 1 S. 4

I will name one reader for every document I create, and I will pick a one-line success metric for each.

Example metric: “First person who can merge a PR within 4 days after onboarding” (cf. Slide 9 Table Row 6).

Week book core: Bhatti Ch. 1 S. 4–21 + Ch. 2 S. 23–44 + Ch. 9 S. 149 + Chinchilla Ch. 1 S. 5–7 + Ch. 6 S. 69–85

Next Week — Week 02

From culture to structure: the standards you will use on your own SRS — Scoping per Chinchilla Ch. 5 S. 53–55.

“Can a 60-page standard describe what two engineers debated for an afternoon?”

Anchor: Bhatti Ch. 2 S. 23–44 (Planning) + Ch. 3 S. 58 „state your most important info first“

We open this question in Week 2 with IEEE 830. By the end of Week 3 you will have answered it on your own project.

PREPARATION · BEFORE NEXT SESSION

Barker Sammlung (W2-Begleittext) + Bhatti Ch. 4 S. 67–81 (Editing)

Read Barker ch. 1 (the lecture's anchor text). Skim IEEE 830-1998 sections 1–4.

Bring two short position notes: one pro and one contra on “Good code documents itself”.

SELF-STUDY · MILESTONE M1

Bhatti Ch. 2 S. 27–29 (READMEs) + Chinchilla Ch. 1 S. 5–7 (Writer-in-the-middle)

Set up Confluence or Notion workspace — four areas (Requirements, Architecture, API, User-Docs), labels, permissions, first template.

Deliverable: URL + screenshot of structure by Friday of Week 1.

Bridge to W2: Bhatti Ch. 2 S. 23–44 + Ch. 4 S. 67–81 + Chinchilla Ch. 5 S. 53–55 + Barker Ch. 1 (Begleittext) + IEEE 830-1998 S. 1–4